



Facultad de Ingeniería
Y Tecnología

Universidad de
Montemorelos



VISIONLAB: PROCESAMIENTO DE IMÁGENES CON OPENCV 5 Y MFC

UNIVERSIDAD DE MONTEMORELOS

MONTEMORELOS N.L.



PRESENTA:

ISAAC RAMSES VILLA ALVAREZ

MATERIA:

PROGRAMACIÓN DEL SISTEMA

PROFESOR:

IGNACIO CRUZ DOMÍNGUEZ

08/10/2026



ÍNDICE

1. Introducción.....	3
1.1 Objetivo.....	3
1.2 Herramientas y versiones.....	3
2. Compilación de OpenCV desde el código fuente.....	4
2.1 ¿Por qué compilar desde el código fuente?.....	4
2.2 Estructura de carpetas.....	4
2.3 Requisitos previos.....	4
2.4 Descarga del código fuente.....	5
2.5 Configuración con CMake.....	5
2.6 Compilación e instalación.....	6
2.7 Verificación de la instalación.....	6
2.8 Script de compilación automatizado.....	7
2.9 Diferencias de OpenCV 5 que afectan al código.....	7
3. El proyecto VisionLab.....	8
3.1 Creación del proyecto en Visual Studio.....	8
3.2 Estructura de archivos.....	8
3.3 Enlazar OpenCV con el proyecto.....	8
3.4 Arquitectura.....	9
4. Uso de OpenCV en la aplicación.....	10
4.1 Filtros implementados.....	10
4.2 Carga y guardado de imágenes.....	10
4.3 Mostrar un cv::Mat en la ventana MFC.....	11
4.4 Interacción: eventos de la interfaz y llamadas a OpenCV.....	11
4.5 Cámara y detección de rostros.....	12
5. Compilar y ejecutar.....	13
5.1 Pasos en Visual Studio.....	13
5.2 La ventana principal.....	13
6. Resultados.....	15
6.1 Capturas de los filtros.....	15
6.2 Pruebas realizadas.....	18
6.3 Rendimiento.....	18
7. Problemas encontrados y soluciones.....	19
8. Conclusiones.....	20
8.1 Trabajo futuro.....	20



1. Introducción

VisionLab es una aplicación de escritorio para Windows escrita en C++17 con **MFC** (Microsoft Foundation Classes) que utiliza la biblioteca de visión por computadora **OpenCV 5** para aplicar filtros y algoritmos de procesamiento de imagen a archivos y a la cámara web. La biblioteca no se descargó como un paquete ya compilado: se obtuvo el código fuente, se configuró con CMake y se compiló con el compilador de Visual Studio, de modo que el proyecto enlaza contra una versión construida en este mismo equipo.

Este documento describe, primero, el proceso de compilación de OpenCV desde el código fuente y, después, el proyecto que la utiliza: cómo se crea y configura en Visual Studio, cómo se comunica la interfaz gráfica con las funciones de OpenCV y qué resultados se obtuvieron al ejecutarlo.

1.1 Objetivo

Compilar OpenCV 5 desde su código fuente, documentar el procedimiento y desarrollar una aplicación MFC que demuestre el uso de la biblioteca y su interacción con una interfaz gráfica (botones, lista de filtros, controles deslizantes, vista previa en tiempo real y cámara).

1.2 Herramientas y versiones

Herramienta	Versión / detalle
OpenCV	5.1.0-dev, rama 5.x (commit 99eb7ce, 07/10/2026)
opencv_contrib	rama 5.x (commit 1275703, 24/09/2026)
CMake	4.2.3
Visual Studio	Visual Studio 2026 Community (generador "Visual Studio 18 2026")
Compilador	MSVC 14.51.36231, conjunto de herramientas v145, x64
Windows SDK	10.0.26100.0
Lenguaje	C++17 (/std:c++17, /utf-8)
Interfaz	MFC como DLL compartida (UseOfMfc = Dynamic), juego de caracteres Unicode
Control de ruta	OpenCV en C:\cv5; proyecto en C:\Documents\Programacion del Sistema\VisionLab

2. Compilación de OpenCV desde el código fuente

2.1 ¿Por qué compilar desde el código fuente?

- **Versión 5.x:** la rama 5.x incluye cambios de la nueva generación de OpenCV (por ejemplo, módulos reorganizados) que todavía no están en los instaladores precompilados estables.
- **Módulos adicionales:** opencv_contrib aporta módulos extra (xobjdetect, ximgproc, face, tracking, etc.) que solo se pueden activar al compilar.
- **Control de opciones:** se decide qué se construye (sin pruebas ni documentación, con FFmpeg, sin OpenCL) y se genera un único opencv_world510.dll.
- **Compatibilidad con el compilador:** la biblioteca se construye con el mismo MSVC y la misma biblioteca de ejecución (/MD) que se usará en el proyecto MFC.



Todo el proceso lo automatiza el script opencv.ps1

Figura 1. Etapas del proceso de compilación de OpenCV y su uso en VisionLab.

2.2 Estructura de carpetas

Todo el trabajo se concentra en C:\cv5. El código fuente de opencv_contrib queda al lado del de OpenCV y la compilación se hace **fuera** del árbol de código (un "out-of-source build"), lo que permite borrar build sin tocar las fuentes.

```

C:\cv5\
├── opencv\           Código fuente de OpenCV (git, rama 5.x)
├── opencv_contrib\  Módulos adicionales (git, rama 5.x)
├── build\           Archivos generados por CMake y objetos de compilación
└── install\        Resultado final: include\, x64\vc18\lib, x64\vc18\bin, etc\
  
```

2.3 Requisitos previos

- **Git**, para clonar los repositorios.
- **CMake** (se usó 4.2.3) disponible en el PATH.
- **Visual Studio** con la carga de trabajo "Desarrollo para el escritorio con C++" (incluye MSVC, MFC y el SDK de Windows).
- **Espacio en disco:** la carpeta build ocupó unos 7.3 GB y install unos 333 MB.



2.4 Descarga del código fuente

Los dos repositorios se clonan con `--depth 1` (solo el último commit, sin todo el historial) y en la rama `5.x`:

```
git clone --branch 5.x --depth 1 ^
https://github.com/opencv/opencv.git C:/cv5/opencv
git clone --branch 5.x --depth 1 ^
https://github.com/opencv/opencv_contrib.git C:/cv5/opencv_contrib
```

Es importante que ambos repositorios pertenezcan a la misma rama: los módulos de `opencv_contrib` se compilan junto con OpenCV y deben corresponder a la misma versión de la API.

2.5 Configuración con CMake

CMake lee `CMakeLists.txt` del código fuente y genera la solución de Visual Studio dentro de `C:\cv5\build`. Las opciones usadas fueron:

```
cmake -G "Visual Studio 18 2026" -A x64 ^
-DCMAKE_CXX_STANDARD=17 -DCMAKE_CXX_STANDARD_REQUIRED=ON ^
-DOPENCV_EXTRA_MODULES_PATH=C:/cv5/opencv_contrib/modules ^
-DBUILD_opencv_world=ON -DBUILD_EXAMPLES=ON -DBUILD_opencv_apps=ON ^
-DBUILD_TESTS=OFF -DBUILD_PERF_TESTS=OFF -DBUILD_DOCS=OFF ^
-DBUILD_opencv_python3=OFF -DBUILD_opencv_java=OFF ^
-DWITH_VTK=OFF -DWITH_OPENCL=OFF -DWITH_TBB=OFF -DWITH_OPENMP=OFF ^
-DWITH_FFMPEG=ON -DBUILD_WITH_DEBUG_INFO=ON ^
-DCMAKE_INSTALL_PREFIX=C:/cv5/install ^
C:/cv5/opencv
```

Opción	Para qué sirve
-G "Visual Studio 18 2026" -A x64	Genera proyectos de Visual Studio 2026 para 64 bits (conjunto de herramientas v145).
CMAKE_CXX_STANDARD=17	Compila con C++17, el mismo estándar que usa el proyecto MFC.
OPENCV_EXTRA_MODULES_PATH	Incorpora los módulos de <code>opencv_contrib</code> a la compilación.
BUILD_opencv_world=ON	Reúne los módulos en una sola biblioteca: <code>opencv_world510.dll</code> y <code>opencv_world510.lib</code> . Simplifica el enlace.
BUILD_TESTS / BUILD_PERF_TESTS = OFF	Omite las pruebas y reduce mucho el tiempo de compilación.
BUILD_EXAMPLES / BUILD_opencv_apps = ON	Construye los ejemplos y herramientas (como <code>opencv_version.exe</code>) para comprobar la instalación.
BUILD_DOCS, python3, java, VTK = OFF	Desactiva componentes que el proyecto no necesita.
WITH_OPENCL, TBB, OPENMP = OFF	Evita dependencias de aceleración que no se usan; se conserva el paralelismo propio de OpenCV.
WITH_FFMPEG=ON	Permite leer y escribir video (se genera <code>opencv_videoio_ffmpeg510_64.dll</code>).



Opción	Para qué sirve
BUILD_WITH_DEBUG_INFO=ON	Incluye símbolos de depuración incluso en la compilación Release.
CMAKE_INSTALL_PREFIX	Carpeta donde se copia el resultado: C:/cv5/install.

Después de configurar se comprueba en CMakeCache.txt que la ruta de contrib quedó registrada (OPENCV_EXTRA_MODULES_PATH:PATH=C:/cv5/opencv_contrib/modules) y que el generador es el esperado.

2.6 Compilación e instalación

```
cd C:\cv5\build
cmake --build . --config Release --parallel <núcleos>
cmake --build . --config Release --target INSTALL --parallel <núcleos>
```

La primera orden compila todos los módulos en modo **Release** usando todos los núcleos disponibles (<núcleos> es el número de procesadores, que el script obtiene con ProcessorCount); la segunda ejecuta el objetivo INSTALL, que copia encabezados, bibliotecas y DLL a C:\cv5\install. En este equipo el proceso completo (clonado, configuración, compilación e instalación) tomó alrededor de 30 minutos (07/10/2026, aproximadamente de 20:00 a 20:32).

2.7 Verificación de la instalación

El resultado se verifica ejecutando opencv_version.exe y revisando que existan los archivos que usará el proyecto:

Archivo	Ubicación	Tamaño
opencv_world510.lib	install\x64\vc18\lib	4.4 MB
opencv_world510.dll	install\x64\vc18\bin	86.7 MB
opencv_videoio_ffmpeg510_64.dll	install\x64\vc18\bin	29.4 MB
Encabezados (opencv2*.hpp)	install\include	—
Clasificadores Haar y LBP (*.xml)	install\etc\haarcascades y lbpcascades	—
opencv_version.exe	install\x64\vc18\bin	0.3 MB

El nombre 510 en los archivos corresponde a la versión 5.1.0. La carpeta vc18 indica que se compiló con la versión 18 de Visual Studio (2026).



2.8 Script de compilación automatizado

Todos los pasos anteriores están automatizados en el script `opencv.ps1` (copia en `scripts\build-opencv5.ps1` dentro del proyecto). Está pensado para poder repetirse desde cero:

Paso	Qué hace el script
1-2	Comprueba que existan <code>git</code> y <code>cmake</code> y que haya una instalación de Visual Studio con las herramientas de C++ (<code>vswhere</code>).
3	Crea las carpetas de trabajo bajo <code>C:\cv5</code> .
4-5	Clona (o actualiza con <code>git pull</code>) <code>opencv</code> y <code>opencv_contrib</code> en la rama 5.x.
6-7	Muestra la rama y el último commit de cada repositorio y verifica que exista <code>opencv_contrib\modules</code> .
8-9	Borra la carpeta <code>build</code> anterior y comprueba que no haya restos de una compilación dentro del código fuente.
10-11	Ejecuta CMake con las opciones de la sección 2.5 y confirma que <code>contrib</code> quedó registrado.
12-13	Compila en Release en paralelo e instala en <code>C:\cv5\install</code> .
14-15	Busca <code>opencv_version.exe</code> , lo ejecuta y muestra un resumen de rutas.

Para ejecutarlo se abre PowerShell y se corre `.\opencv.ps1`. Si algún paso falla, el script se detiene con un mensaje (`$ErrorActionPreference = "Stop"`), por lo que no queda una instalación a medias sin aviso.

2.9 Diferencias de OpenCV 5 que afectan al código

Al usar una versión nueva fue necesario ajustar el código respecto a lo que aparece en la mayoría de tutoriales (que usan OpenCV 4). Estos cambios se detectaron revisando los encabezados instalados y con el compilador:

Elemento	OpenCV 4	OpenCV 5.1 (esta instalación)
<code>cv::CascadeClassifier</code> (detector Haar/LBP)	<code>opencv2/objdetect.hpp</code>	<code>opencv2/xobjdetect.hpp</code> (módulo <code>contrib xobjdetect</code>)
<code>cv::contourArea</code> , <code>cv::boundingRect</code>	<code>opencv2/imgproc.hpp</code>	<code>opencv2/geometry.hpp</code> (módulo <code>geometry</code>)
Nombre de la biblioteca	<code>opencv_world4xx.lib</code>	<code>opencv_world510.lib</code>
Carpeta de bibliotecas	<code>x64\vc16 / vc17</code>	<code>x64\vc18</code>



3. El proyecto VisionLab

3.1 Creación del proyecto en Visual Studio

El proyecto es una **aplicación MFC basada en cuadro de diálogo** escrita en C++. La solución se creó en la carpeta C:\Documents\Programacion del Sistema\VisionLab con estos ajustes:

- **Tipo:** aplicación de escritorio de Windows con MFC (cuadro de diálogo principal, sin vistas ni documentos).
- **Uso de MFC:** biblioteca compartida (DLL), con lo que el ejecutable es pequeño.
- **Juego de caracteres:** Unicode (cadenas wchar_t, CString de 16 bits).
- **Plataforma:** x64, conjunto de herramientas v145, estándar C++17.
- **Encabezado precompilado:** pch.h, donde se incluyen MFC y los encabezados de OpenCV para acelerar la compilación.

3.2 Estructura de archivos

VisionLab\	
— VisionLab.sln / VisionLab.vcxproj	Solución y proyecto de Visual Studio
— ImageProcessor.h / .cpp	Lógica de OpenCV (sin MFC)
— VisionLabDlg.h / .cpp	Ventana principal y eventos
— VisionLab.h / .cpp	Clase de la aplicación (CWinApp)
— VisionLab.rc / resource.h	Diseño del cuadro de diálogo
— pch.h / pch.cpp, framework.h	Encabezado precompilado
— scripts\	build-opencv5.ps1, build-visionlab.ps1
— docs\	Ejemplo de imagen guardada por la app
— bin\x64\Release\	VisionLab.exe + DLL de OpenCV (al compilar)

3.3 Enlazar OpenCV con el proyecto

OpenCV se integra con cuatro ajustes en VisionLab.vcxproj (equivalentes a Propiedades del proyecto en Visual Studio):

Ajuste	Valor	Función
C/C++ > Directorios de inclusión adicionales	C:\cv5\install\include	Para que <code>#include <opencv2/...></code> encuentre los encabezados.
Vinculador > Directorios de bibliotecas	C:\cv5\install\x64\vc18\lib	Dónde buscar el <code>.lib</code> .
Vinculador > Dependencias adicionales	opencv_world510.lib	Biblioteca de importación de la DLL.
Eventos posteriores a la compilación	xcopy de DLL y XML	Copia <code>opencv_world510.dll</code> , <code>opencv_videoio_ffmpeg510_64.dll</code> y <code>haarcascade_frontalface_default.xml</code> junto al <code>.exe</code> .

La ruta de OpenCV se define una sola vez en la propiedad `OpenCV5Root` al inicio del `.vcxproj`; si la biblioteca se mueve, solo cambia esa línea. Además se agregan `/utf-8` (los acentos de los textos se guardan en UTF-8) y `MultiThreadedDLL (/MD)`, la misma biblioteca de ejecución con la que se compiló OpenCV.

Solo existe la configuración Release | x64. OpenCV se compiló únicamente en Release, por lo que no existe una `opencv_world510d.lib`; mezclar un proyecto en Debug (que usa el CRT de depuración) con una biblioteca Release no es válido. Si se necesitara depurar con OpenCV en Debug, habría que repetir la compilación de la sección 2 con la configuración Debug.

3.4 Arquitectura

El código se separa en dos capas. `ImageProcessor` contiene **solo OpenCV** y no conoce nada de MFC; `CVisionLabDlg` contiene la interfaz y se limita a llamar a `ImageProcessor`. Entre ambas solo viajan objetos `cv::Mat` y números enteros (los valores de los sliders).

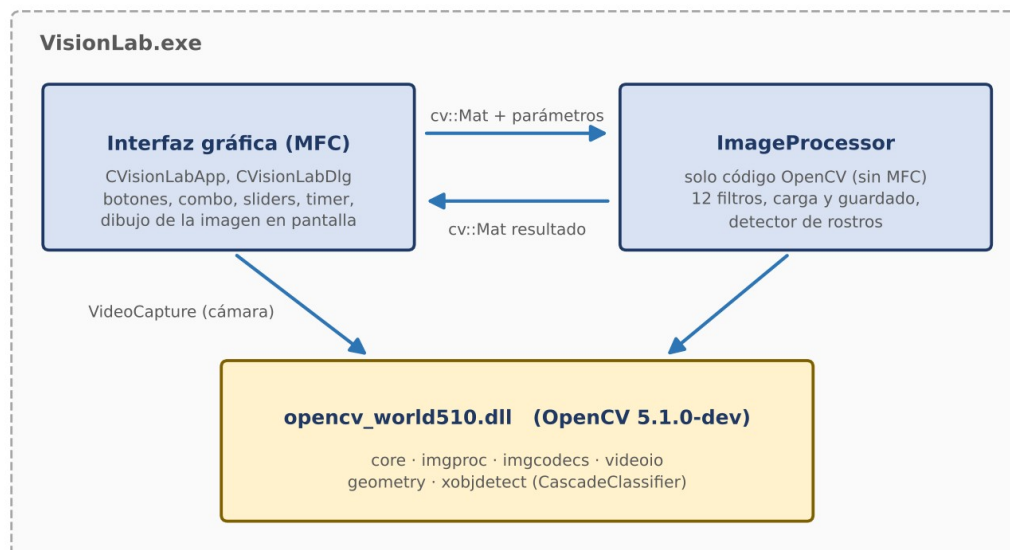


Figura 2. Arquitectura de *VisionLab*: la interfaz MFC usa *ImageProcessor*, que a su vez usa *OpenCV*.

Esta separación facilita probar la lógica de visión de forma independiente (durante el desarrollo, el código de `ImageProcessor` se pudo verificar con un compilador distinto, sin necesitar la ventana) y agregar un filtro nuevo solo requiere un valor en el enum, una fila en la tabla de filtros y un case en `Apply()`.

4. Uso de OpenCV en la aplicación

4.1 Filtros implementados

La lista desplegable de la ventana se llena automáticamente a partir de una tabla (kFilters) que describe cada filtro y los nombres, rangos y valores iniciales de sus dos parámetros. Al elegir un filtro, la ventana reconfigura los sliders con esos datos.

Filtro	Funciones de OpenCV	Parámetros
Original	Mat::clone	—
Escala de grises	cvtColor (BGR → GRAY → BGR)	—
Negativo	bitwise_not	—
Brillo y contraste	Mat::convertTo(α , β)	Contraste %, brillo
Desenfoque gaussiano	GaussianBlur	Radio del kernel (tamaño 2r+1)
Ecualización de histograma	cvtColor (YCrCb), split, equalizeHist, merge	—
Sepia	transform (matriz 3×3), addWeighted	Intensidad %
Umbralización	threshold / adaptiveThreshold	Umbral; modo adaptativo 0/1
Bordes (Canny)	GaussianBlur, Canny	Umbral bajo y alto
Caricatura	medianBlur, adaptiveThreshold, bilateralFilter, bitwise_and	Detalle de bordes, suavizado
Pixelado	resize (INTER_AREA y luego INTER_NEAREST)	Tamaño de bloque
Contornos	Canny, dilate, findContours, contourArea, boundingRect, drawContours	Umbral Canny, área mínima
Detección de rostros	CascadeClassifier (Haar), equalizeHist, detectMultiScale	Vecinos mínimos, escala

4.2 Carga y guardado de imágenes

cv::imread y cv::imwrite reciben el nombre del archivo como std::string, lo que puede fallar con caracteres no ASCII en rutas de Windows. Por eso la aplicación lee y escribe los bytes con _wfopen_s (rutas Unicode) y delega a OpenCV solo la codificación:

```
// Cargar: bytes del archivo -> cv::Mat (siempre BGR de 8 bits y 3 canales)
std::vector<uchar> buf = /* bytes leídos con _wfopen_s */;
cv::Mat img = cv::imdecode(buf, cv::IMREAD_COLOR);

// Guardar: cv::Mat -> bytes PNG/JPG/BMP según la extensión elegida
std::vector<uchar> out;
cv::imencode(".png", img, out);
```

Para imágenes grandes la ventana trabaja con una **vista previa de máximo 1280 píxeles** para que los sliders respondan al instante; al guardar, el filtro se vuelve a aplicar a la imagen original a resolución completa.

4.3 Mostrar un cv::Mat en la ventana MFC

Los dos recuadros de imagen son controles estáticos "dibujados por el propietario" (SS_OWNERDRAW). Cuando Windows pide repintarlos, OnDrawItem convierte el cv::Mat en un mapa de bits de Windows. Los detalles importantes son:

- cv::Mat guarda los píxeles en orden **B-G-R**, que es justo el orden de un DIB de 24 bits de Windows; no hace falta convertir canales.
- Las filas de un DIB deben ser múltiplo de 4 bytes; por eso el ancho de la imagen que se dibuja se ajusta a un múltiplo de 4 ($tw \&= \sim 3$).
- La imagen se escala conservando la proporción (bandas negras alrededor) y se dibuja primero en un mapa de bits en memoria para evitar parpadeo.
- Un alto negativo en BITMAPINFOHEADER indica que las filas vienen de arriba hacia abajo, como en cv::Mat.

```
double s = std::min((double)w / img.cols, (double)h / img.rows); // escala
int tw = std::max(4, (int)(img.cols * s)) & ~3; // ancho múltiplo
de 4
int th = std::max(1, (int)(img.rows * s));
cv::Mat view; cv::resize(img, view, cv::Size(tw, th), 0, 0, cv::INTER_AREA);

BITMAPINFO bi = {};
bi.bmiHeader.biSize = sizeof(BITMAPINFOHEADER);
bi.bmiHeader.biWidth = tw;
bi.bmiHeader.biHeight = -th; // filas de arriba hacia abajo
bi.bmiHeader.biPlanes = 1;
bi.bmiHeader.biBitCount = 24; // B-G-R, igual que cv::Mat
SetDIBitsToDevice(hdc, x, y, tw, th, 0, 0, 0, th, view.data, &bi, DIB_RGB_COLORS);
```

4.4 Interacción: eventos de la interfaz y llamadas a OpenCV

Control	Evento MFC	Qué ocurre
Lista de filtros	CBN_SELCHANGE	Reconfigura los sliders (rango, valor inicial, etiquetas) y reprocesa.
Sliders	WM_HSCROLL	Actualiza la etiqueta con el valor y llama a Process().
Abrir imagen	BN_CLICKED	CFileDialog → imdecode → muestra original y resultado.
Guardar resultado	BN_CLICKED	CFileDialog → vuelve a filtrar a resolución completa → imencode → archivo.
Iniciar / detener cámara	BN_CLICKED + WM_TIMER	VideoCapture abre la cámara; un temporizador de 33 ms lee un cuadro y lo procesa (~30 fps).



Control	Evento MFC	Qué ocurre
Espejo	casilla	<code>cv::flip</code> horizontal de cada cuadro de la cámara.

El corazón de la interacción es `Process()`: toma la imagen de trabajo, llama a `ImageProcessor::Apply` con el filtro y los valores de los sliders, mide el tiempo con `cv::getTickCount` y pide repintar los dos recuadros.

```
void CVisionLabDlg::Process()
{
    int64 t0 = cv::getTickCount();
    try {
        m_dst = m_proc.Apply(m_src, (FilterType)m_combo.GetCurSel(),
                             m_slider1.GetPos(), m_slider2.GetPos());
    } catch (const cv::Exception& e) { // un error de OpenCV no cierra la app
        SetDlgItemText(IDC_STATIC_INFO, L"Error de OpenCV: " + CString(e.what()));
        return;
    }
    m_lastMs = (cv::getTickCount() - t0) * 1000.0 / cv::getTickFrequency();
    UpdateStatus(); // barra de estado
    GetDlgItem(IDC_PIC_ORIG)->Invalidate(FALSE);
    GetDlgItem(IDC_PIC_RES)->Invalidate(FALSE);
}
```

4.5 Cámara y detección de rostros

La cámara se abre con `cv::VideoCapture` usando el backend `DirectShow` (`CAP_DSHOW`), que abre mucho más rápido en Windows que el backend por defecto. Cada cuadro se convierte en la imagen de trabajo y pasa por el mismo flujo que una imagen de archivo.

La detección de rostros usa un clasificador en cascada de Haar: la imagen se pasa a grises, se ecualiza el histograma y `detectMultiScale` devuelve los rectángulos de cada rostro, que se dibujan con `cv::rectangle`. El archivo `haarcascade_frontalface_default.xml` se copia junto al `.exe` al compilar; si no está, la aplicación lo busca en `C:\cv5\install\etc\haarcascades` y, si tampoco existe, muestra un aviso sobre la imagen en lugar de cerrarse.

5. Compilar y ejecutar

5.1 Pasos en Visual Studio

- Abrir `VisionLab.sln` (Archivo > Abrir > Proyecto o solución).
- Verificar que la barra superior diga **Release** y **x64**.
- Menú **Compilar** > **Compilar solución**. La carpeta de salida es `bin\x64\Release`.
- Ejecutar con **Depurar** > **Iniciar sin depurar** (o abrir `VisionLab.exe` desde el Explorador).

Alternativa por línea de comandos: `.\scripts\build-visionlab.ps1 -Run` localiza MSBuild con `vswhere`, compila en Release x64 y abre la aplicación.

```
Salida
Mostrar salida de:  Compilación
Compilación iniciada a las 10:27 p. m....
1>----- Operación Compilar iniciada: Proyecto: VisionLab, configuración: Release x64 -----
1> pch.cpp
1> ImageProcessor.cpp
1> VisionLab.cpp
1> VisionLabOlg.cpp
1> Generando código...
1> VisionLab.vcxproj -> C:\Documents\Programacion del Sistema\VisionLab\bin\x64\Release\VisionLab.exe
1> Copiando DLL de OpenCV y clasificador de rostros...
1> 1 archivo(s) copiado(s)
1> 1 archivo(s) copiado(s)
1> 1 archivo(s) copiado(s)
***** Compilación: 1 correcto, 0 erróneo, 0 actualizado, 0 omitido *****
***** Compilar completado a las 10:27 p. m. y tardó 03.599 segundos *****
```

Figura 3. Salida de Visual Studio: compilación correcta, 0 errores, y copia de las DLL de OpenCV.

5.2 La ventana principal

La ventana tiene una barra superior (abrir, guardar, cámara, espejo y lista de filtros), dos recuadros de imagen (original a la izquierda y resultado de OpenCV a la derecha), dos sliders con sus etiquetas y una barra de estado que muestra la versión de OpenCV, el tamaño de la imagen, el filtro activo y el tiempo de procesamiento. Al iniciar se carga una imagen de demostración dibujada con las funciones de OpenCV (`circle`, `rectangle`, `ellipse`, `putText`), así los filtros se pueden probar sin abrir ningún archivo.

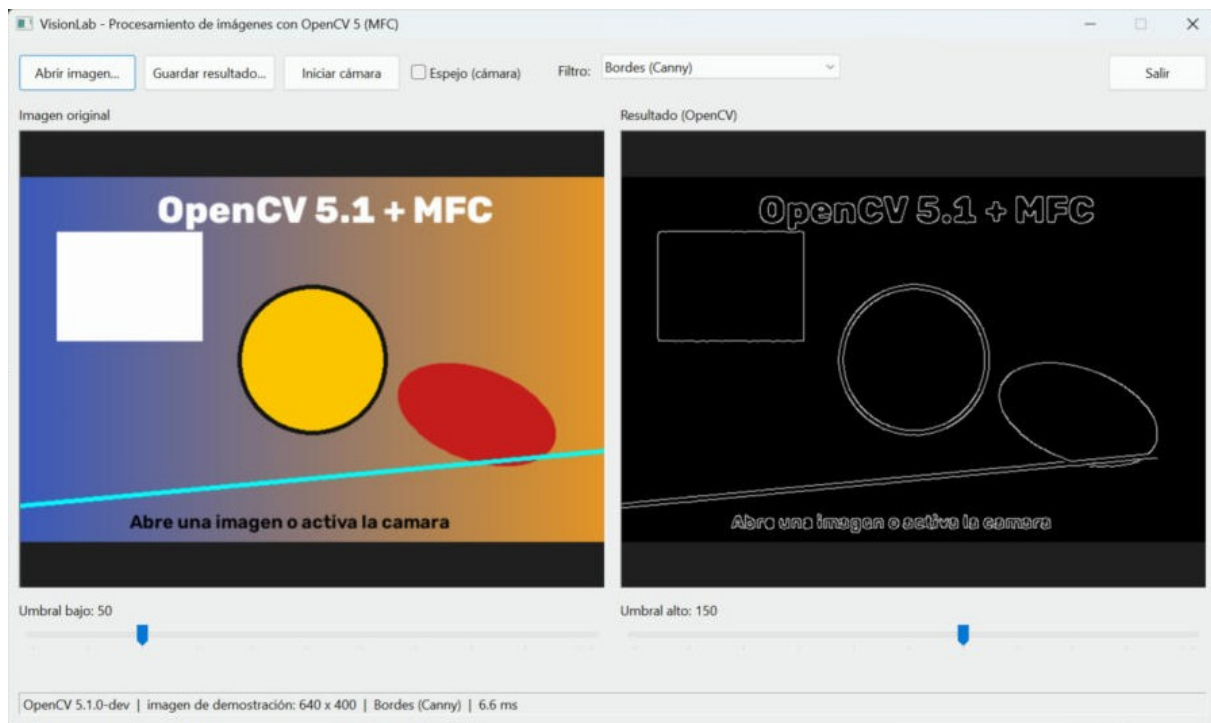


Figura 4. Ventana de VisionLab con el filtro Bordes (Canny): OpenCV 5.1.0-dev, 6.6 ms.

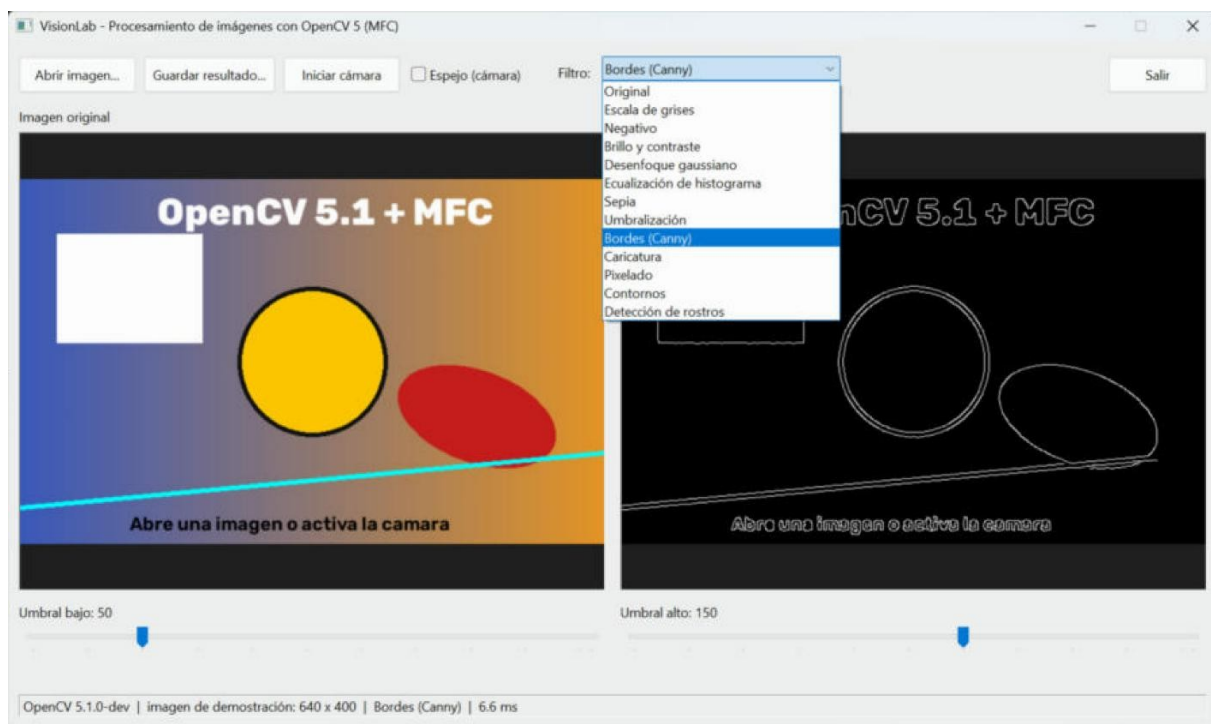


Figura 5. Lista de filtros disponibles; al elegir uno cambian las etiquetas y rangos de los sliders.

6. Resultados

6.1 Capturas de los filtros

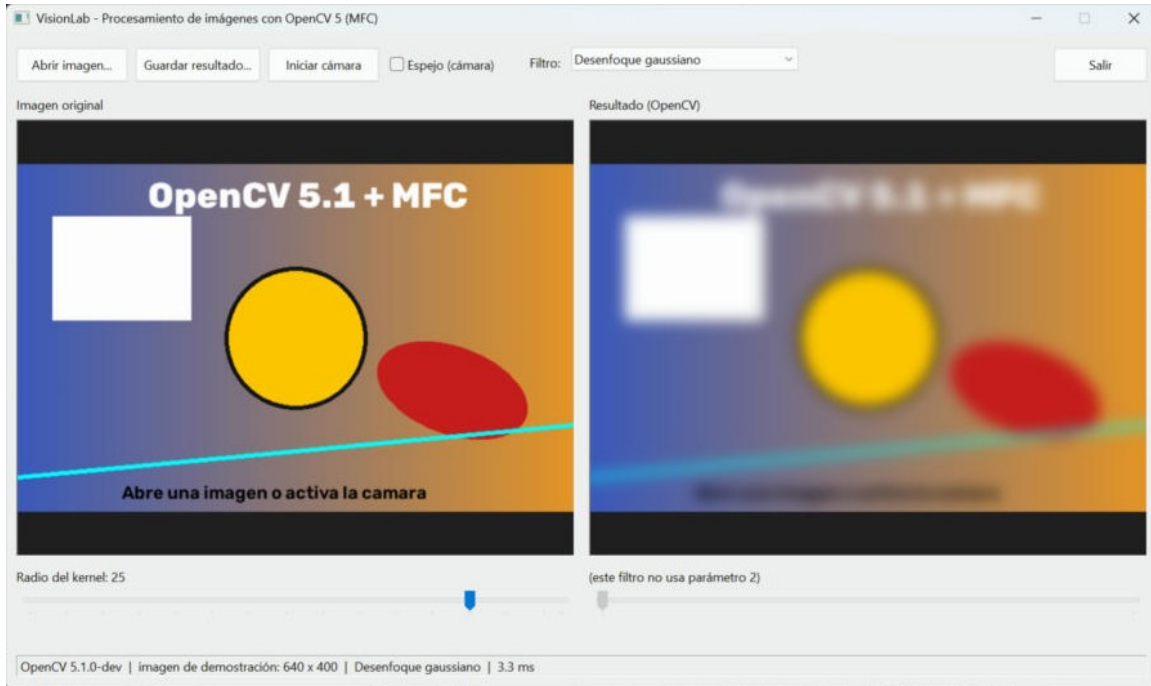


Figura 6. Desenfoque gaussiano con radio 25 (slider movido): 3.3 ms.

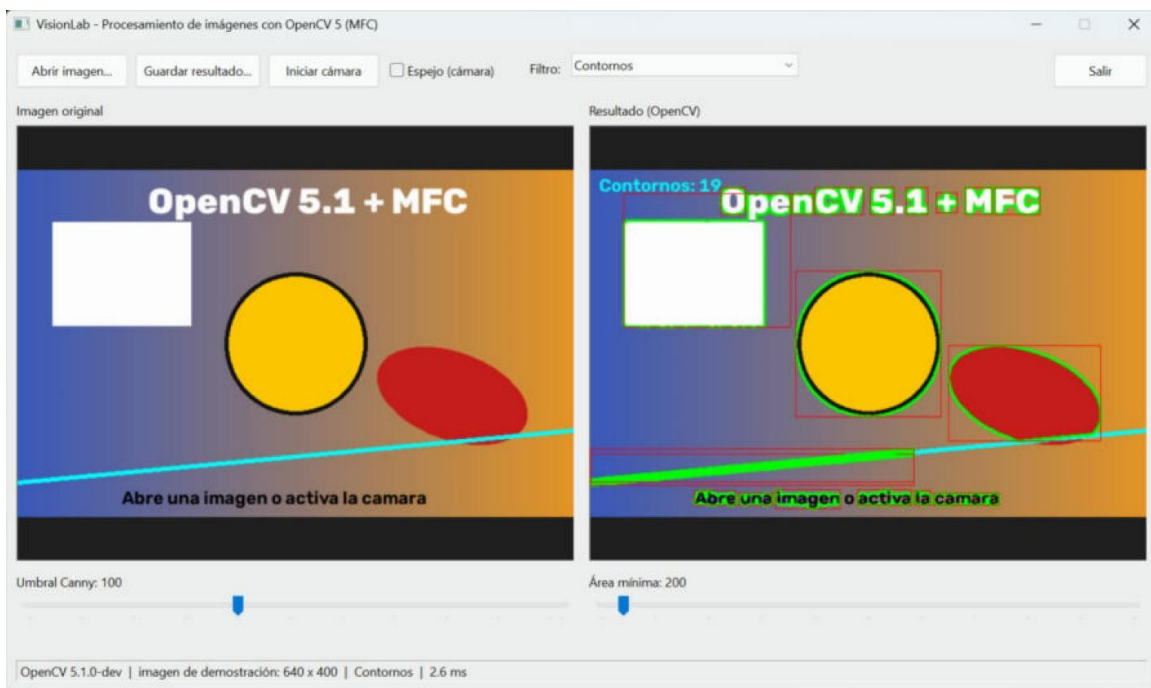


Figura 7. Contornos: 19 contornos detectados, con su caja delimitadora (rojo) y contorno (verde); 2.6 ms.

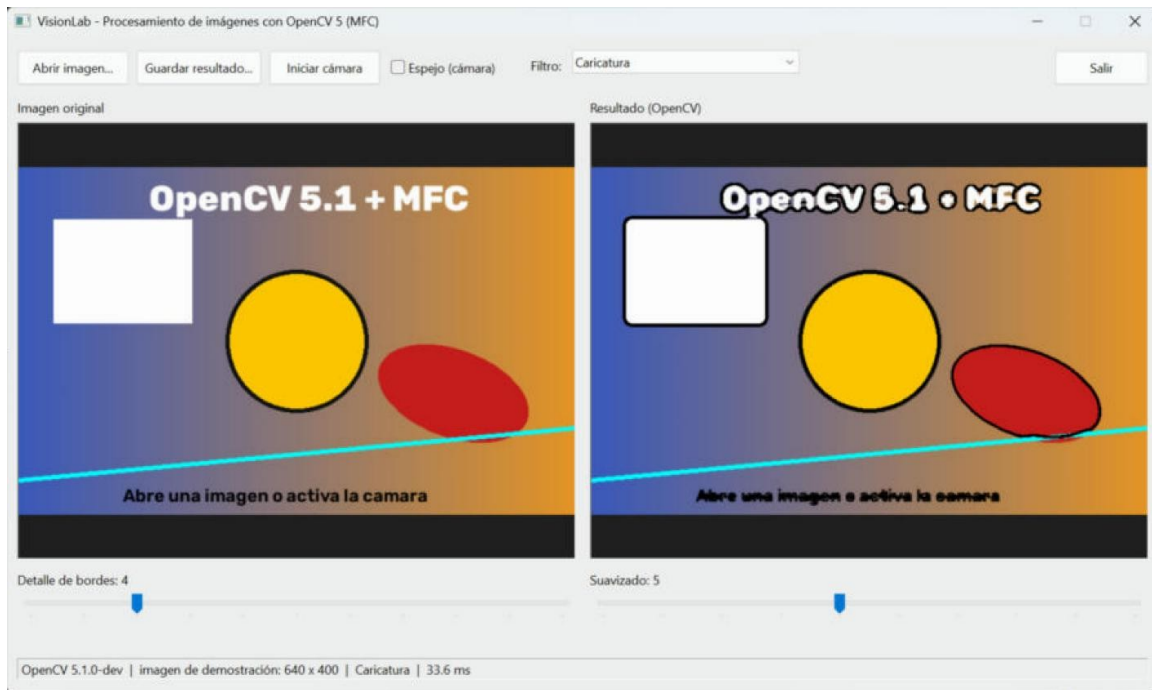


Figura 8. Caricatura (bordes por umbral adaptativo + filtro bilateral).

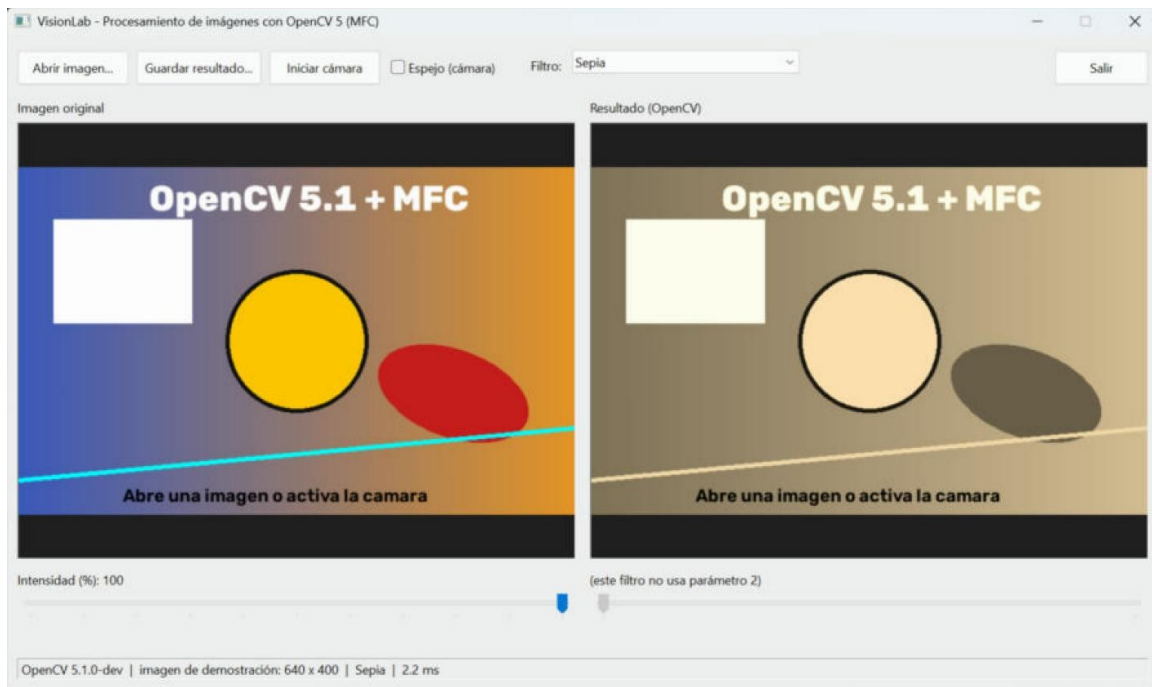


Figura 9. Sepia con intensidad 100 %.

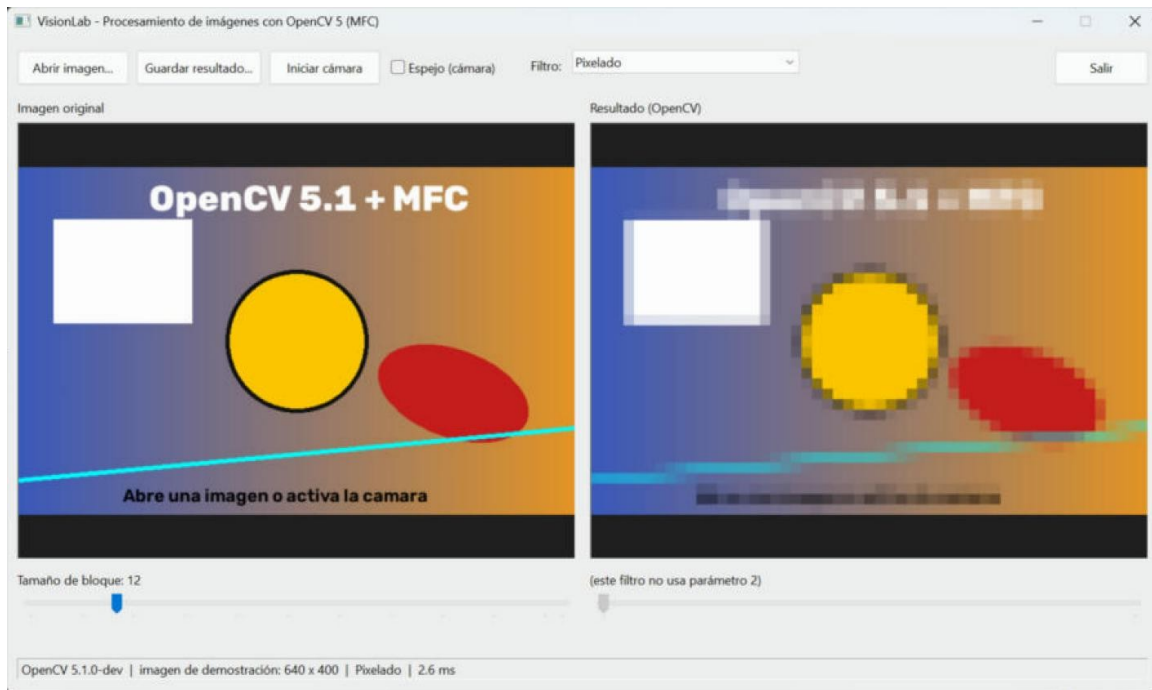


Figura 10. Pixelado con bloques de 12 píxeles.

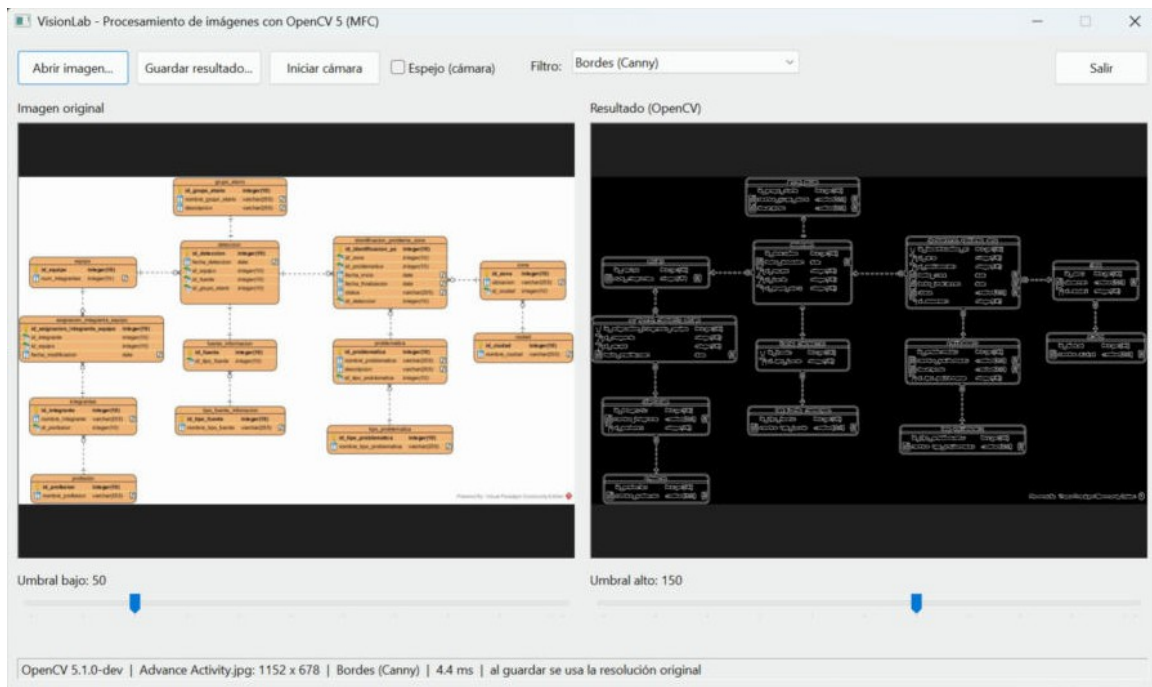


Figura 11. Imagen cargada desde un archivo (1152×678) con el filtro Canny: 4.4 ms.



6.2 Pruebas realizadas

La aplicación se compiló y se ejecutó en el equipo de desarrollo (Windows, Visual Studio 2026). Se probó lo siguiente:

Prueba	Resultado
Compilación Release x64	Correcta: 0 errores, 0 advertencias.
Inicio de la aplicación	Abre la ventana, muestra la imagen de demostración y "OpenCV 5.1.0-dev" en la barra de estado.
Cambio de filtro y de sliders	Canny 5.8–6.7 ms, caricatura 20–34 ms, desenfoque 3.3 ms, contornos 2.6 ms (640×400). Las etiquetas y rangos de los sliders cambian con el filtro.
Abrir imagen desde archivo	Correcto con un JPG de 1152×678 (Canny en 4.4 ms).
Guardar resultado	Genera un PNG válido de 1152×678 en la carpeta docs (a resolución completa).
Cámara web	Abre la cámara a 640×480 y procesa cuadros en tiempo real (contornos en 1.5–2.7 ms).
Detección de rostros	Detecta 1 rostro en 5.6 ms con la cámara.
Cierre	La aplicación termina con código 0.

6.3 Rendimiento

Todos los filtros trabajan sobre la vista previa (máximo 1280 px) y tardan unos pocos milisegundos, por lo que la cámara a ~ 30 cuadros por segundo no se ve afectada. El filtro más costoso es la caricatura (filtro bilateral), con 20–34 ms en la imagen de 640×400 , aún adecuado para imágenes fijas.

7. Problemas encontrados y soluciones

Durante la primera compilación real aparecieron los errores siguientes; se documentan porque son típicos al enlazar una biblioteca propia con un proyecto de Visual Studio.

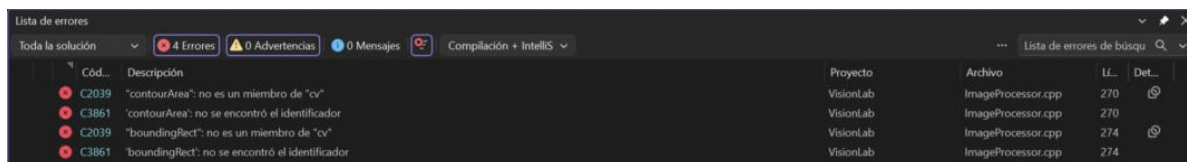


Figura 12. Error de compilación C2039 / C3861: `contourArea` y `boundingRect` no están en `cv` de OpenCV 5.

Problema	Causa	Solución
C1083: no se puede abrir el archivo incluir: opencv2/core.hpp	El directorio de inclusión no se estaba aplicando; la ruta de OpenCV se tomaba de una propiedad condicionada a una variable de entorno (probablemente ya definida en el equipo con otro valor).	Se definió una propiedad propia con ruta fija, <code>OpenCV5Root = C:\cv5\install</code> , en el <code>.vcxproj</code> .
C2039 / C3861: contourArea y boundingRect no son miembros de cv	En OpenCV 5 estas funciones pasaron del módulo <code>imgproc</code> al módulo <code>geometry</code> .	Se agregó <code>#include <opencv2/geometry.hpp></code> .
CascadeClassifier no se encuentra en objdetect.hpp	En OpenCV 5 la clase está en el módulo contrib <code>xobjdetect</code> . Se detectó al revisar los encabezados antes de compilar.	Se incluyó <code><opencv2/xobjdetect.hpp></code> ; funciona porque <code>opencv_world510</code> incluye los módulos contrib.
Tras detener la cámara, la barra de estado seguía diciendo "imagen de demostración"	El último cuadro de la cámara queda como imagen de trabajo, pero la etiqueta de origen no se actualizaba.	Al detener la cámara se cambia la etiqueta a "captura de cámara".
No se puede depurar en Debug	OpenCV se compiló solo en Release; no existe <code>opencv_world510d.lib</code> .	El proyecto solo ofrece Release x64 (ver sección 3.3).



8. Conclusiones

Se compiló OpenCV 5.1 con sus módulos contrib desde el código fuente, con un proceso repetible gracias al script `opencv.ps1`, y se usó en una aplicación MFC real: VisionLab. El proyecto muestra la integración completa entre una interfaz gráfica de Windows y una biblioteca de visión por computadora: eventos de la ventana que disparan procesamiento con `cv::Mat`, resultados dibujados en pantalla, lectura y escritura de imágenes, y captura en tiempo real desde la cámara.

Lo más valioso del ejercicio fue comprobar en la práctica que usar una versión de desarrollo implica revisar los encabezados instalados en lugar de confiar en ejemplos de versiones anteriores (varias funciones cambiaron de módulo), y que separar la lógica de visión de la interfaz facilita tanto la verificación como la ampliación del proyecto.

8.1 Trabajo futuro

- Compilar también OpenCV en Debug para poder depurar paso a paso dentro de las funciones de OpenCV.
- Agregar un histograma en vivo y la grabación de video con VideoWriter.
- Permitir redimensionar la ventana y guardar/restaurar la configuración de filtros.
- Probar el detector de rostros basado en redes neuronales (FaceDetectorYN) incluido en `objdetect`.